

Walkthrough textual archivo por archivo - FitZone

Proyecto: app-gym-main / FitZone **Objetivo:** explicar que contiene cada archivo propio del repositorio y dejar evidencia concreta con lineas o metadatos. **Alcance:** se revisan archivos fuente, configuracion, datos, documentacion y entregables generados. Se excluyen dependencias externas (vendor/, node_modules/) y assets compilados/binarios (public/build/, build/, imagenes y videos) porque son salidas o paquetes de terceros, no codigo fuente propio.

1. Entrada de la aplicacion y enrutamiento

Archivo	Que contiene	Evidencia
public/index.php	Es el punto de entrada HTTP. Carga el bootstrap de la app, importa controladores y crea el router.	Carga includes/app.php en la linea 3, importa controladores en las lineas 5-9 y crea new Router() en la linea 11.
public/index.php	Declara las rutas principales de la aplicacion: landing, login, logout, recuperacion, registro, admin y usuario.	Ruta / en linea 15, login en lineas 19-20, logout en lineas 22-23, registro en lineas 34-35, admin en lineas 39-41 y usuario en lineas 45-48.
public/index.php	Ejecuta la resolucion final de rutas.	Llama a \$router->comprobarRutas() en la linea 51.
Router.php	Define la clase MVC\Router y mantiene arreglos de rutas GET y POST.	Namespace en linea 3, clase en linea 5, propiedades \$getRoutes y \$postRoutes en lineas 7-8.
Router.php	Registra rutas GET y POST.	Metodo get() en lineas 10-13 y metodo post() en lineas 15-18.
Router.php	Inicia sesion, obtiene URL/metodo actual y decide que callback ejecutar.	session_start() en lineas 24-25, URL/metodo en lineas 33-34, seleccion de ruta en lineas 36-40 y call_user_func() en linea 45.
Router.php	Renderiza vistas dentro del layout general.	Extrae datos en lineas 54-57, abre buffer en linea 59, incluye vista en linea 62 e incluye layout en linea 64.

2. Controladores

Archivo	Que contiene	Evidencia
controllers/PrincipalController.php	Controlador de la pagina publica principal.	Clase PrincipalController en linea 6 y metodo principal() que renderiza auth/principal en lineas 7-8.
controllers/LoginController.php	Controlador del flujo de login. Recibe POST, crea un Usuario, valida credenciales, busca por email y verifica password.	Usa Usuario y Router en lineas 4-5; procesa POST en linea 12; crea new Usuario(\$_POST) en linea 13; valida login en linea 14; busca usuario en linea 17; verifica password en linea 21.
controllers/LoginController.php	Crea variables de sesion y redirige segun rol.	Escribe \$_SESSION['id'], nombre, email y login en lineas 25-28; evalua \$usuario->admin === 1 en linea 31; redirige a /inicio-admin en linea 33 y a /inicio-user en linea 37.
controllers/LoginController.php	Implementa logout.	Verifica/inicia sesion en lineas 60-62, limpia \$_SESSION en linea 64, destruye sesion en linea 65 y redirige a / en linea 67.
controllers/LoginController.php	Maneja pantalla de olvido de password, pero todavia como placeholder.	Metodo olvide() en lineas 71-81; en POST agrega mensaje de no implementado en lineas 74-76.
controllers/LoginController.php	Tiene un stub de recuperacion.	Metodo recuperar() imprime Desde recuperar en lineas 84-85.
controllers/LoginController.php	Maneja creacion de cuenta.	Metodo crearCuenta() comienza en linea 88; crea new Usuario en linea 89; procesa POST en linea 92; llama \$usuario->sincronizar(\$_POST) en linea 94; valida nueva cuenta en linea 95; guarda usuario en linea 111.

Archivo	Que contiene	Evidencia
controllers/InicioAdminController.php	Controlador de vistas administrativas.	Clase en línea 6; inicioAdmin() renderiza auth/inicio-admin en líneas 8-9; administrarPlanes() y administrarClases() renderizan la misma vista en líneas 12-17.
controllers/InicioUserController.php	Controlador de vistas del usuario autenticado.	Clase en línea 6; inicioUser() renderiza auth/inicio-user en líneas 8-9; planes(), suscripciones() y clases() renderizan la misma vista en líneas 12-21.

3. Modelos y acceso a datos

Archivo	Que contiene	Evidencia
models/ActiveRecord.php	Clase base para persistencia MongoDB.	Namespace Model en línea 3; imports de BulkWrite, Manager y Query en líneas 5-8; clase ActiveRecord en línea 10.
models/ActiveRecord.php	Guarda conexión, nombre de base de datos, colección y alertas compartidas.	Propiedades \$db, \$databaseName, \$collection y \$alertas en líneas 12-15; setDB() en líneas 17-20.
models/ActiveRecord.php	Implementa consultas genéricas.	where() en líneas 35-38, all() en líneas 40-42, find() en líneas 44-47 y get() en líneas 49-51.
models/ActiveRecord.php	Ejecuta consultas MongoDB y convierte documentos a objetos.	Valida existencia de DB en líneas 53-57, crea Query en líneas 59-64, ejecuta executeQuery() en línea 67 y mapea documentos en líneas 73-78.
models/ActiveRecord.php	Implementa crear, actualizar y eliminar documentos.	guardar() decide crear/actualizar en líneas 106-112; crear() usa BulkWrite en líneas 118-151; actualizar() en líneas 154-173; eliminar() en líneas 176-191.
models/ActiveRecord.php	Construye el namespace database.collection.	collectionNamespace() en líneas 194-196 y collectionName() en líneas 198-200.
models/Usuario.php	Modelo de usuario para la colección usuarios.	Clase en línea 5 y colección por defecto en línea 6.
models/Usuario.php	Define propiedades de usuario.	id, nombre, apellido, email, admin, token y password en líneas 8-14.
models/Usuario.php	Inicializa el objeto desde un arreglo.	Constructor en líneas 16-25 asigna id, nombre, apellido, email, admin, token y password.
models/Usuario.php	Valida registro y login.	validarNuevaCuenta() en líneas 29-37 y validarLogin() en líneas 39-44.
models/Usuario.php	Verifica duplicados, password y hashing.	existeUsuario() en líneas 48-55, verificarPassword() en líneas 57-60 y hashPassword() en líneas 62-64.
models/Usuario.php	Genera token y define que atributos se persisten.	crearToken() en líneas 66-68; atributos() devuelve campos persistentes en líneas 70-78.
models/Usuario.php	Permite cambiar el nombre de la colección por variable de entorno.	collectionName() lee MONGODB_USERS_COLLECTION en líneas 81-83.

4. Bootstrap, helpers y base de datos

Archivo	Que contiene	Evidencia
includes/app.php	Bootstrap de la aplicación.	Requiere helpers, autoload de Composer y conexión DB en líneas 3-5.
includes/app.php	Injecta conexión MongoDB en ActiveRecord.	Importa Model\ActiveRecord en línea 8 y llama ActiveRecord::setDB(\$db, \$mongoDatabase) en línea 9.
includes/database.php	Crea la conexión MongoDB.	Usa MongoDB\Driver\Manager en línea 3; inicializa \$db = null en línea 5; crea new Manager(\$mongoUri, ...) en líneas 20-25.

Archivo	Que contiene	Evidencia
includes/database.php	Lee variables de entorno y maneja errores de configuracion.	Valida extension MongoDB en lineas 7-10; lee MONGODB_URI y MONGODB_DATABASE en lineas 12-13; retorna si falta URI en lineas 15-17; registra excepcion en lineas 26-28.
includes/funciones.php	Contiene helper de debug.	debuguear() imprime var_dump y termina ejecucion en lineas 3-8.
includes/funciones.php	Contiene helper de escape/sanitizacion HTML.	Funcion s() usa htmlspecialchars() en lineas 10-13.

5. Vistas PHP

Archivo	Que contiene	Evidencia
views/layout.php	Layout HTML compartido para todas las vistas.	Estructura html, head y body en lineas 1-19; carga Google Fonts en lineas 8-11; carga build/css/app.css en linea 13; imprime \$contenido en linea 16.
views/templates/alertas.php	Partial que renderiza alertas.	Recorre \$alertas en lineas 2-3; imprime clase y mensaje en lineas 6-7.
views/auth/login.php	Vista de inicio de sesion.	Logo en lineas 1-4; titulo en lineas 5-7; incluye alertas en lineas 10-11; formulario POST en lineas 14-28; campos email/password en lineas 17 y 21.
views/auth/login.php	Enlaces auxiliares y decoracion SVG.	Enlaces a crear cuenta y olvide en lineas 23-25; divisor SVG en lineas 30-36.
views/auth/crear-cuenta.php	Vista de registro.	Titulo en lineas 5-6; incluye alertas en lineas 9-10; formulario POST a /crear-cuenta en linea 13.
views/auth/crear-cuenta.php	Campos de registro con re-render de valores escapados.	Campos nombre, apellido, email y password en lineas 16, 20, 24 y 28; usa s() en lineas 16, 20 y 24.
views/auth/olvide-password.php	Vista de recuperacion de password.	Titulo en lineas 5-6; incluye alertas en lineas 9-10; formulario POST a /olvide en linea 13; campo email en linea 16.
views/auth/inicio-admin.php	Dashboard placeholder de administrador.	Navegacion admin en lineas 1-11; enlaces a administrar planes/clases en lineas 5-6; logout en linea 9.
views/auth/inicio-admin.php	Hero con video para admin.	Texto en lineas 14-19; video MP4/WebM en lineas 21-24.
views/auth/inicio-user.php	Dashboard placeholder de usuario.	Navegacion de usuario en lineas 1-12; enlaces a Planes, Suscripciones y Clases en lineas 5-7; logout en linea 10.
views/auth/inicio-user.php	Hero con video para usuario.	Texto en lineas 15-20; video MP4/WebM en lineas 22-25.
views/auth/principal.php	Landing page publica.	Navbar con enlaces internos en lineas 2-13; link a login en linea 12.
views/auth/principal.php	Seccion hero con video.	Hero #inicio en lineas 17-28; texto principal en lineas 20-21; fuentes de video en lineas 25-26.
views/auth/principal.php	Seccion "Sobre nosotros".	Contenedor #sobreNosotros en lineas 30-38; texto en lineas 31-33; imagen en linea 36.
views/auth/principal.php	Seccion de servicios.	Contenedor #servicios en lineas 40-71; bloques de entrenamiento, nutricion, clases y cardio en lineas 42-69.
views/auth/principal.php	Seccion contacto y footer.	Mapa embebido en lineas 73-77; formulario de contacto en lineas 79-96; footer en lineas 102-104.

6. Frontend fuente: JavaScript y SCSS

Archivo	Que contiene	Evidencia
src/js/app.js	Archivo JavaScript fuente actualmente vacio.	wc -l reporto 0 lineas para src/js/app.js; nl -ba no produjo contenido.
src/scss/app.scss	Punto de entrada Sass.	Importa modulos base y layout en lineas 1-2.
src/scss/base/_index.scss	Agregador de parciales base.	Reexporta normalize, variables, mixins, globales y tipografia en lineas 1-5.
src/scss/base/_variables.scss	Variables globales de tipografia, breakpoints, colores y pesos.	Fuente principal en lineas 1-2; breakpoints en lineas 4-7; colores en lineas 9-16; separacion y pesos en lineas 18-24.
src/scss/base/_mixins.scss	Mixins de media queries, boton y grid.	Mixins telefono, tablet, desktop en lineas 4-18; boton() en lineas 21-45; grid() en lineas 47-51.
src/scss/base/_globales.scss	Estilos globales HTML/body/contenedor/titulos/imagenes.	Reset de box sizing en lineas 3-11; body en lineas 13-20; .contenedor en lineas 29-32; titulos e imagenes en lineas 38-57.
src/scss/base/_tipografia.scss	Reglas tipograficas base.	Aplica fuente, margen, peso y color a h1, h2, h3 en lineas 3-9; tamanos en lineas 11-19; parrafos en lineas 21-24.
src/scss/base/_normalize.scss	Copia de normalize.css para normalizar estilos del navegador.	Archivo de 349 lineas segun conteo; se reexporta desde _index.scss linea 1.
src/scss/layout/_index.scss	Agregador de parciales de layout.	Reexporta UI, contacto, footer, header, servicios, sobreNosotros, video, login y alertas en lineas 1-11.
src/scss/layout/_UI.scss	Parcial UI vacio o reservado.	Solo importa variables y mixins en lineas 1-2; no define reglas adicionales.
src/scss/layout/_header.scss	Estilos de navegacion.	.nav-main en lineas 4-42; .nav-menu en lineas 9-12; logo en lineas 14-17; enlaces en lineas 19-30.
src/scss/layout/_contacto.scss	Estilos de seccion contacto y formulario.	.contacto grid en lineas 4-9; .mapa, .contact-form, .form en lineas 11-24; inputs en lineas 26-34; boton submit en lineas 41-52.
src/scss/layout/_footer.scss	Estilos del footer.	.foot en lineas 4-18; fondo verde, altura y centrado en lineas 5-9; parrafo interno en lineas 11-17.
src/scss/layout/_servicios.scss	Estilos de tarjetas/secciones de servicios.	.servicios en lineas 4-6; grid de 4 columnas en lineas 8-10; estilos de bloques en lineas 24-37; iconos span en lineas 39-42.
src/scss/layout/_sobreNosotros.scss	Estilos de seccion sobre nosotros.	Grid de dos columnas en lineas 4-8; bloque de texto en lineas 9-12; color/alineacion en lineas 14-17.
src/scss/layout/_video.scss	Estilos del hero/video.	.video con posicion y alto en lineas 4-8; texto del hero en lineas 10-28; overlay gradiente en lineas 31-42; video en lineas 44-47.
src/scss/layout/_login.scss	Estilos de pantallas de login/registro/olvide.	.contenedor - login en lineas 4-36; logo en lineas 8-15; texto en lineas 18-34; campos en lineas 37-52; acciones en lineas 54-66; divisor SVG en lineas 69-88.
src/scss/layout/_alertas.scss	Estilos de mensajes de alerta.	.alerta en lineas 4-9; variante error en lineas 11-13; variante exito en lineas 14-16.

7. Build, dependencias y configuracion local

Archivo	Que contiene	Evidencia
composer.json	Configuracion Composer y autoload PSR-4.	Nombre/descripcion en lineas 2-3; autoload PSR-4 para MVC, Controllers y Model en lineas 5-10; require vacio en linea 18.

Archivo	Que contiene	Evidencia
composer.lock	Lockfile Composer generado. No tiene paquetes instalados por Composer.	_readme indica archivo generado en lineas 2-6; packages y packages-dev vacios en lineas 8-9.
package.json	Configuracion npm para frontend.	Nombre/version en lineas 2-3; scripts gulp y build en lineas 6-9; dependencias dev en lineas 16-35.
package-lock.json	Lockfile npm con versiones exactas de dependencias frontend.	Nombre/version/lockfile en lineas 2-5; dependencias dev raiz en lineas 11-30; ejemplo de paquete transitorio @babel/runtime en lineas 32-42.
gulpfile.js	Pipeline Gulp para compilar CSS, JS e imagenes.	Imports de Gulp/plugins en lineas 1-17; rutas fuente en lineas 19-23; tarea CSS en lineas 25-33; JS en lineas 35-40; imagenes/WebP en lineas 42-53; watch en lineas 56-60; exports en lineas 63-67.

8. Docker, Azure y CI/CD

Archivo	Que contiene	Evidencia
Dockerfile	Imagen PHP 8.2 Apache para ejecutar la app.	Base php:8.2-apache en linea 1; instala librerias y extension MongoDB en lineas 4-19; habilita mod_rewrite en linea 22.
Dockerfile	Configura DocumentRoot e instala dependencias Composer.	WORKDIR en linea 25; APACHE_DOCUMENT_ROOT a /var/www/html/public en linea 28; copia app en linea 36; composer install en linea 39; expone puerto 80 en linea 41.
docker-compose.yml	Compose local para levantar la app.	Servicio app en lineas 1-2; build local en lineas 3-5; puerto 8080:80 en lineas 7-8; volumen del repo en lineas 9-10; variables MongoDB en lineas 11-14.
docker-compose.azure.yml	Compose para usar imagen publicada en ACR.	Imagen desde \${ACR_LOGIN_SERVER} en linea 3; puerto en lineas 4-5; variables MongoDB en lineas 6-9.
.dockerignore	Excluye archivos del contexto Docker.	Excluye .git, .DS_Store, node_modules, src, compose, SQL y vendor/bin en lineas 1-7.
azure/acr-build.sh	Script para construir imagen en Azure Container Registry.	set -euo pipefail en linea 3; valida argumentos en lineas 5-8; asigna variables en lineas 10-13; ejecuta az acr build en lineas 15-19.
azure/appsettings.example.env	Plantilla de variables para Azure App Service.	WEBSITES_PORT=80 en linea 1; MONGODB_URI via Key Vault en linea 2; base/coleccion en lineas 3-4; timeouts en lineas 5-7.
.github/workflows/ci-cd.yml	Workflow de GitHub Actions para CI/CD.	Se dispara en push/PR a main/master en lineas 3-11; define PHP/Node/image en lineas 16-19; job CI en lineas 21-64; job deploy en lineas 65-119.
.github/workflows/ci-cd.yml	Pasos de validacion y despliegue.	Valida Composer en lineas 37-38; instala Composer en lineas 40-41; lint PHP en lineas 43-44; npm ci en lineas 52-53; build frontend en lineas 55-56; build Docker en lineas 58-63; deploy Azure en lineas 115-119.

9. Datos de prueba y persistencia

Archivo	Que contiene	Evidencia
database.mongodb.json	Seed MongoDB con dos usuarios de QA.	Arreglo JSON inicia en linea 1; usuario admin en lineas 2-10 con admin: 1 en linea 9; usuario normal en lineas 11-19 con admin: 0 en linea 18.
database.mongodb.json	Passwords seed ya hasheados.	Hash bcrypt del admin en linea 7 y hash bcrypt del usuario normal en linea 16.
database.sql	Esquema SQL legado para tabla usuarios.	CREATE TABLE usuarios en lineas 1-9 con campos nombre, apellido, email, password, token y admin.
database.sql	Inserts de usuarios de prueba SQL.	Comentarios de usuario admin en lineas 11-13; insert admin en lineas 14-15; insert usuario normal en lineas 17-19.

10. Documentacion y entregables generados

Archivo	Que contiene	Evidencia
README.md	Documentacion principal del proyecto.	Describe FitZone y stack en lineas 1-12; layout del proyecto en lineas 14-24; requisitos en lineas 26-33; setup local en lineas 35-91.
README.md	Datos de QA, rutas y limitaciones conocidas.	Usuarios QA en lineas 93-100; rutas principales en lineas 102-111; limitaciones conocidas en lineas 113-119; workflow de assets en lineas 125-133.
AZURE_DEPLOYMENT_GUIDE.md	Guia de despliegue Azure.	Objetivo Azure/Cosmos DB en lineas 1-3; recursos requeridos en lineas 5-13; app settings en lineas 29-37; Key Vault en lineas 38-52; flujo de despliegue en lineas 82-93.
lista_chequeo_seguridad.md	Checklist de inspeccion de seguridad.	Objetivo en lineas 1-5; instrucciones de uso en lineas 7-22; alcance en lineas 24-30; autenticacion en lineas 32-43; autorizacion en lineas 45-55; sesiones/CSRF en lineas 57-79.
informe_walkthrough_inspeccion.md	Informe previo de walkthrough e inspeccion.	Titulo y metadata en lineas 1-6; resumen ejecutivo en lineas 8-14; alcance de archivos en lineas 16-43; metodologia walkthrough/inspeccion en lineas 45-69; arquitectura observada en lineas 71-80.
informe_inspeccion_walkthrough_2026-05-30.md	Informe formal con defect log, walkthrough y matriz ISO 25010.	Informacion de inspeccion en lineas 3-12; defect log en lineas 14-29; metricas en lineas 46-59; evidencia de comandos en lineas 61-79.
INFORME_Formato_INFORMES_FitZone.docx	Documento Word adaptado al formato del PDF de clase: inspeccion formal y walkthrough.	Metadato de archivo: ls -lh reporto 40K y fecha 2026-05-30 16:05. Al ser .docx, no tiene lineas de texto plano confiables sin extraer OOXML.
Informe_Inspeccion_Walkthrough_FitZone.docx	Documento Word narrativo con lenguaje mas natural sobre inspeccion y walkthrough.	Metadato de archivo: ls -lh reporto 42K y fecha 2026-05-30 16:01.

11. Archivos y carpetas excluidos del walkthrough fuente

Elemento	Motivo
vendor/	Dependencias de Composer generadas/instaladas; no son codigo propio del proyecto.
node_modules/	Dependencias npm instaladas; el contenido real se controla con package-lock.json.
public/build/ y build/	Assets compilados por Gulp; la fuente esta en src/.
Imágenes y video (*.png, *.jpg, *.webp, *.mp4)	Assets binarios sin lineas de codigo; se referencian desde vistas y SCSS.

12. Lectura general del sistema

El proyecto sigue una estructura MVC simple. La peticion entra por `public/index.php`, se registra en `Router.php`, se despacha a un controlador en `controllers/`, consulta datos mediante `models/` y renderiza una vista en `views/` dentro del layout global. La base de datos esperada es MongoDB/Cosmos DB, configurada por variables de entorno y conectada en `includes/database.php`. El frontend se compila desde `src/scss` y `src/js` hacia `public/build` usando Gulp.

La evidencia principal de esa arquitectura esta distribuida asi: rutas en `public/index.php` lineas 15-51; resolucion/render en `Router.php` lineas 20-64; bootstrap en `includes/app.php` lineas 3-9; conexion MongoDB en `includes/database.php` lineas 12-25; modelo base MongoDB en `models/ActiveRecord.php` lineas 53-191; vistas en `views/auth/*.php`; y pipeline frontend en `gulpfile.js` lineas 19-67.